USB 2.0 Verification Plan

Revision 0.3

USB 2.0 Documentation

AUTHORS

Shruti Patel

Sneha patil

Chandan B V

Pavan Kumar P

Table of contents

Page No.

Table of Figures

Table of Tables

Revision History

Version	Date	Modified by	Changes
0.1	24/12/2025	Shruti Patel, Sneha Patil, Chandan, Pavan Kumar P	Initial Changes
0.2	16/02/2026	Shruti Patel, Sneha Patil, Chandan,	Updated the TB Architecture
0.3	20/05/2026	Shruti Patel, Sneha Patil, Chandan,	Updated Testcases
0.4	27/05/2026	Shruti Patel, Sneha Patil, Chandan,	Updated Coverage
0.5	4/06/2026	Shruti Patel, Sneha Patil, Chandan,	Updated Assertions

1.0 INTRODUCTION

Universal Serial Bus an industry standards that defines the cables, connectors, and communication protocols used for connection, communication, and power supply between computers and electronic devices. The USB communication is always initiated by a Host and responded by a Device.To simplify and standardize the connection of peripheral devices to computers and other hosts.

This document will provide you the information and reference documents. The verification process and given the brief introduction on the USB Verification plan and TB operations of the USB 2.0.

It provides the scope of UVM Testbench Architecture of USB 2.0. It includes functional coverage and assertions along with test scenarios.

2.0 USB 2.0 OVERVIEW

USB – Universal Serial Bus. First designed to allow connections to the PC without expansion cards the USB became a communication standard for almost all electronic devices.

2.1 Earlier Versions of USB:

- USB 1.0: Specified data rates of 1.5Mbit/s (Low Bandwidth)
- USB 1.1: Specified data rates of 12Mbits/s.
- USB 2.0: Added high maximum bandwidth of 480Mbits/s.
- USB 3.0/3.1: Having higher data rates of 5GBit/s called as Super speed.

2.2 Different Speeds for USB 2.0:

2.2.1 Low Speed: 1.5Mbits/s.
2.2.2 Full Speed: 12Mbits/s.
2.2.3 High Speed: 480Mbits/s.

2.3 USB Connectors

2.3.1 USB Type-A: The classic rectangular connector.

2.3.2 USB Type-B: A square-shaped USB connector commonly used in printers and other peripherals.

2.3.3 Micro-USB: Small connector, common on older smartphones and accessories.

2.3.4 USB Type-C: The modern standard reversible plug, versatile, supports high power and data.

2.4 USB Applications

List of examples of USB connects: Data storage (Flash drives, external hard drives), Human Interface Devices (Keyboard, Mouse, Game controllers), Imaging (Webcams, Scanners, Printers), Charging (Smartphones, Tablets, Laptops), Audio/Video (Headsets).

2.5 Features of USB

2.4.3 Plug and Play: Devices are automatically detected and configured by the operating systems.
2.4.4 Hot Plugging: Devices can be connected or disconnected while the computer is running without rebooting.
2.4.5 High Speed: Supports fast data transfer rates.
2.4.6 Single Standard: One type of interfaces for many devices (keyboards, mice, printers, cameras, etc...)
2.4.7 Power and Data in one cable: The host gives 5V power and also transfers data through the same cable.
2.4.8 Host-controlled: Devices cannot talk to each other directly, everything goes through the host.

USB system is described by three definitional areas:

2.4.9 USB Host
2.4.10 USB Devices
2.4.11 USB interconnect

3.0 USB HOST:

- There is only one host in USB system.
- Host controls all communication on the USB bus.
- It decides which device communicates, when it does, and how power is distributed.
- The host supplies power to the device and requests data as needed.

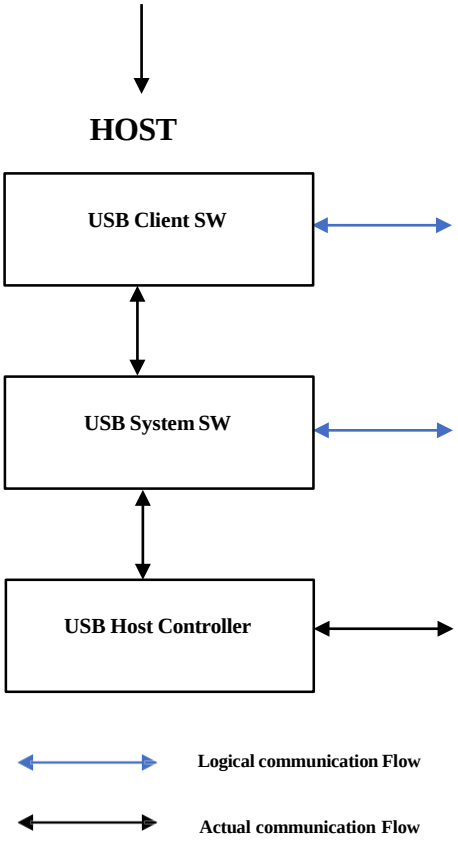

Fig No 3.1 Host Composition

The host's logical composition includes:

- USB Client Software
- USB System Software
- USB Host Controller

3.1 USB Client Software:

- USB client software runs on the host system. Interfaces with USB System Software to communicate with a specific USB Device.
- This is the application uses a USB device.
- It doesn't talk directly to hardware, instead it make requests through higher-level APIs provided by operating system.

3.2 USB System Software:

- USB system software acts as a bridge between client software and a hardware.
- It includes the OS's USB stack, device drivers and class drivers. It manages enumeration (detecting and configuring devices when plugged in).
- Loads the correct driver based on device descriptors.
- Implements Control, Bulk, Interrupt and Isochronous transfers.

3.3 USB Host Controller:

- The USB Host controller is responsible for managing communication between host and the USB Device.
- It is the hardware circuit inside the host that physically controls USB signaling.
- It sends tokens, schedules transfers.
- Works with system software via a Host Controller Driver (HCD).

4.0 USB DEVICE:

- USB Device is any peripheral/ piece of hardware that connects to a host using USB cables and connectors.
- Each device provides structured information (device descriptors, configuration descriptor, interface descriptors, endpoint descriptors) so the host knows what the device is and how to use it.
- Devices can draw power from the USB port.

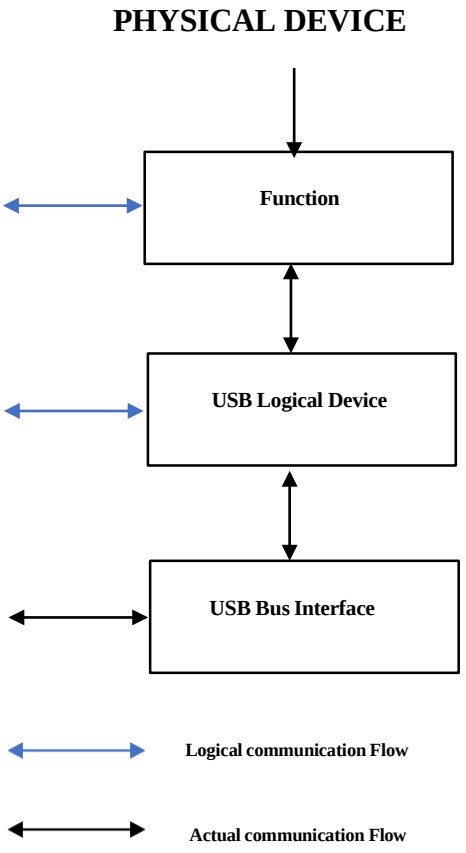

Fig No 4.1 Physical Device Composition

The Device's logical composition includes:

- USB Bus interface
- USB logical device
- Functions

4.1 USB Bus interface:

- USB bus interface is the lowest layer of the device that connects physical and electrical connection between the device and the USB host.
- It handles signaling (D+ and D- differential lines).
- It manages packet framing, bit stuffing, sync patterns, CRC checks.
- It provides power interface (drawing current from VBUS).

4.2 USB Logical Device:

- USB logical device defines device descriptors (Vendor ID, product ID, class) and provides configuration, interface, and endpoint descriptors.
- Manages addressing, each device gets a unique address during enumeration.
- Implements control endpoint (endpoint 0) for setup and configuration requests.

4.3 USB Function:

- Function is the actual operational logic of the USB device.
- Each function is implemented using one or more endpoints.
- It handles class-specific or vendor-specific requests.
- It manages data flow through bulk, interrupt or isochronous endpoints.

5.0 USB BUS TOPOLOGY

The USB connects USB devices with the USB host. The USB physical interconnect is a tiered star topology. A hub is at the centre of each star. Each wire segment is a point-to-point connection between the host and a hub or function, or a hub connected to another hub or function.

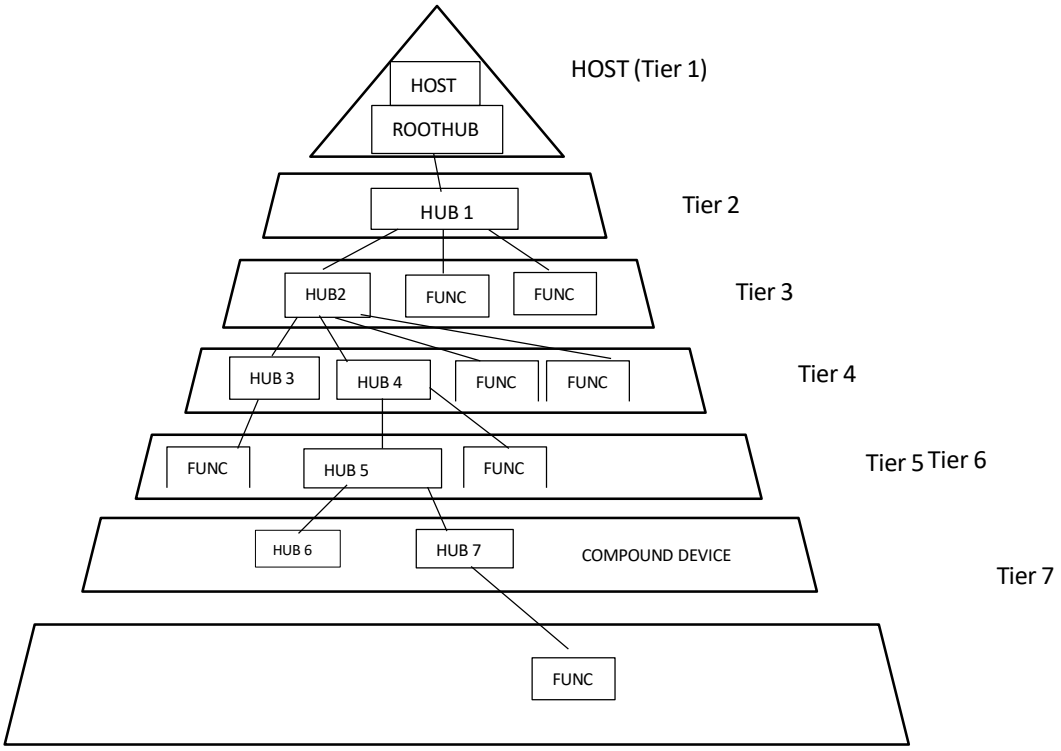

Fig No 5.1 USB Topology

Due to timing constraints allowed for hub and cable propagation times, the maximum number of tiers allowed is seven (including the root tier). Note that in seven tiers, five non-root hubs maximum can be supported in a communication path between the host and any. device. A compound device occupies two tiers therefore, it cannot be enabled if attached at tier level seven. Only functions can be enabled in tier seven.

6.0 USB COMMUNICATION FLOW

The USB provides a communication service between software on the host and its USB function. Functions can have different communication flow requirements for different client-to-function interactions. The USB provides better overall bus utilization by allowing the separation of the different communication flows to a USB function. Each communication flow makes use of some bus access to accomplish communication between client and function.

Each communication flow is terminated at an endpoint on a device. Device endpoints are used to identify aspects of each communication flow.

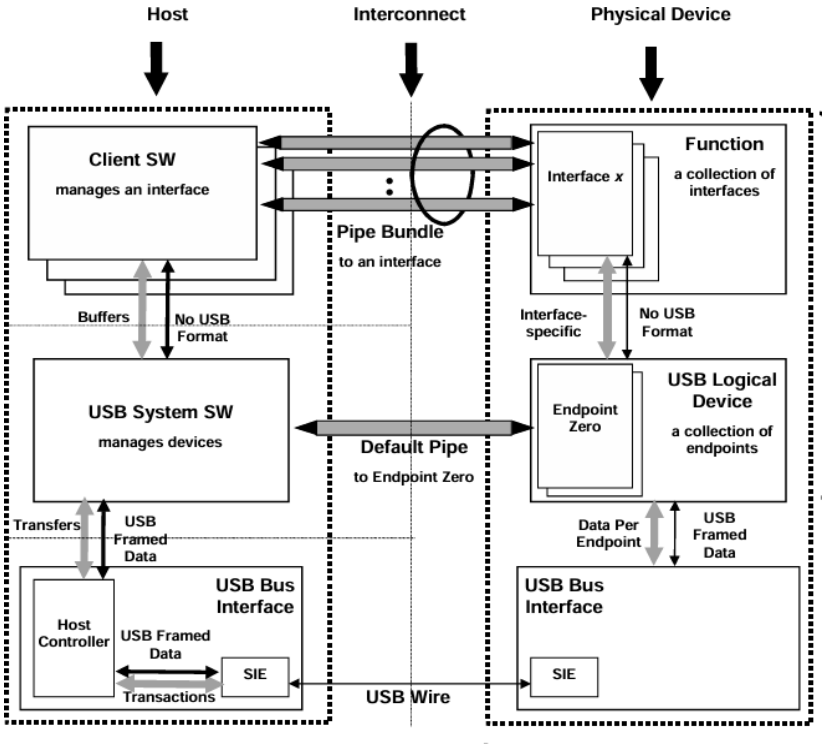

Fig.No 6.1 USB Communication flow

7.0 USB ARCHITECTURE

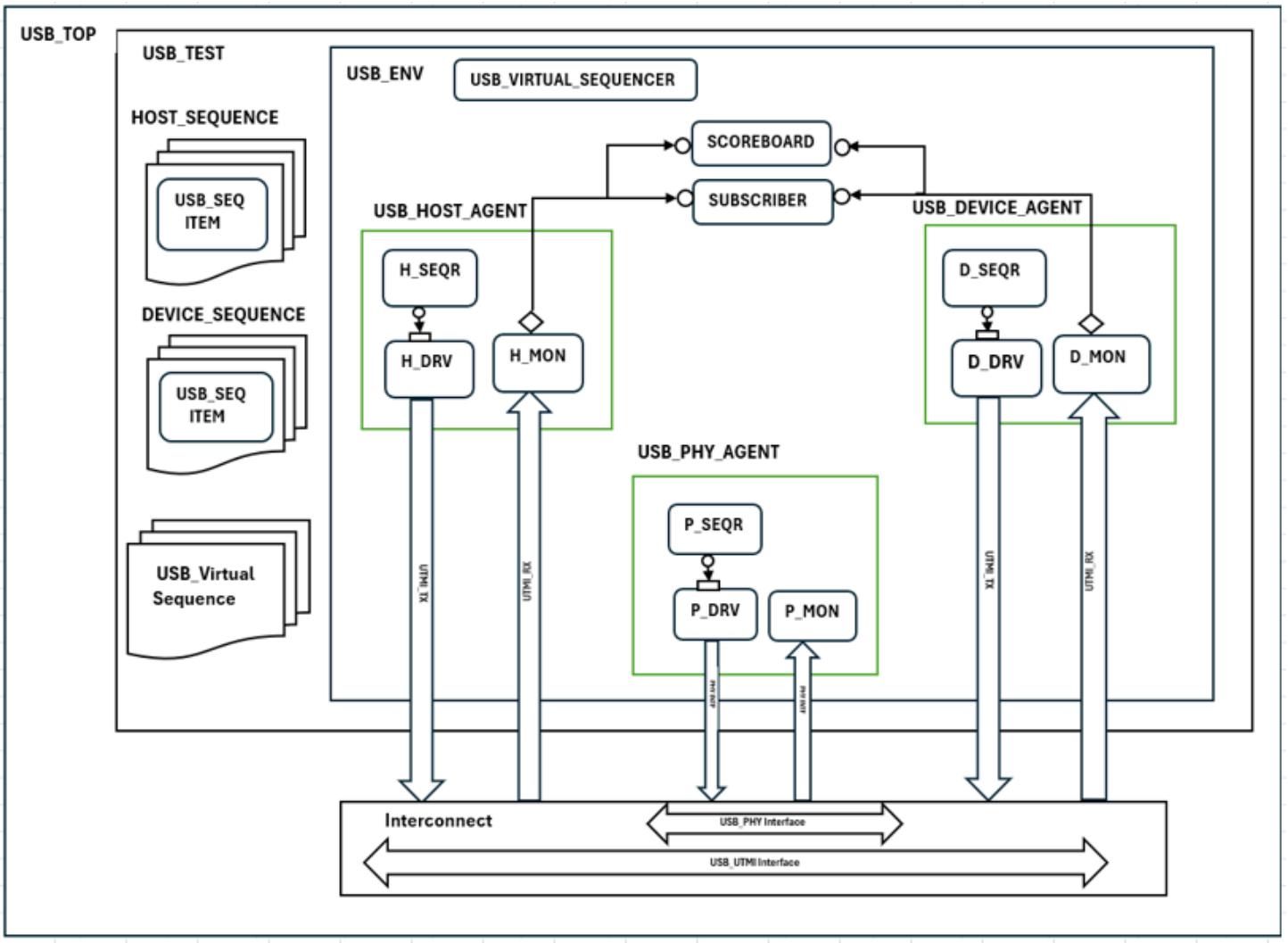

Fig.No 7.1 USB architecture

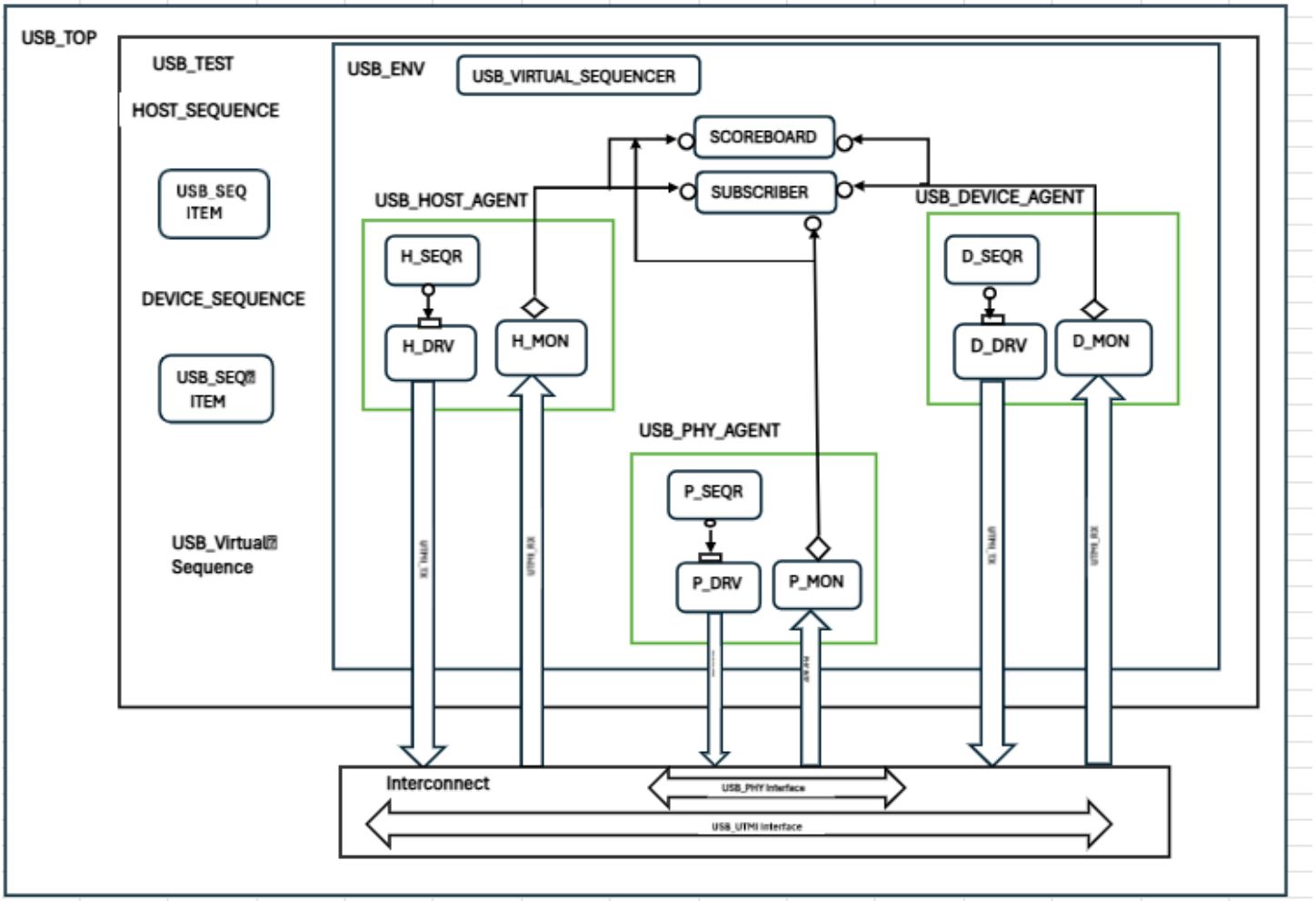

Fig.No 7.2 USB Architecture

8.0 TEST BENCH DESCRIPTION

8.1 TOP

The Top Module acts as the static container for the complete USB 2.0 VIP environment. It contains the instantiation of all required interfaces, clock generation logic, reset generation, and interconnection logic between Host, Device, and PHY interfaces. The top module generates clocks based on the required USB 2.0 operating speeds such as Low Speed (LS), Full Speed (FS), and High Speed (HS). The interface handles are stored in the uvm_config_db using the set() method and retrieved by lower hierarchy components using the get() method. The top module also contains the USB Host Interface, USB Device Interface, and USB PHY/UTMI Interface which are connected internally for packet transmission and reception. The simulation starts by calling the run_test() method from the top module.

8.2 TEST

The Test is the top-level UVM component derived from the uvm_test class. It is responsible for configuring and constructing the complete USB 2.0 VIP environment. Different functional scenarios such as control transfer, bulk transfer, interrupt transfer, isochronous transfer, enumeration,protocol error conditions are verified through different tests. The test starts the Virtual Sequence which controls Host, Device, and PHY sequences through the Virtual Sequencer.

8.3 ENVIRONMENT

The Environment is derived from uvm_env and acts as a container for all verification components present in the USB 2.0 VIP. The environment contains the USB Host Controller Agent, USB Device Controller Agent, USB PHY Agent, Virtual Sequencer, Scoreboard, and Subscriber. All components are created during the build_phase() and connected during the connect_phase(). The monitors from different agents communicate with the Scoreboard and Subscriber using analysis ports. The Environment provides complete connectivity and synchronization between all protocol-level and PHY-level verification components.

8.4 USB VIRTUAL SEQUENCER

The USB Virtual Sequencer is derived from uvm_sequencer and is responsible for coordinating all lower-level sequencers present inside the Host Controller Agent, Device Controller Agent, and PHY Agent. The Virtual Sequencer contains handles for the Host Sequencer, Device Sequencer, and PHY Sequencer. It enables synchronization between Host-side, Device-side, and PHY-level operations during USB protocol verification. The Virtual Sequencer controls simultaneous execution of multiple sequences and ensures proper transaction ordering across the complete USB 2.0 verification environment.

8.5 USB VIRTUAL SEQUENCE

The USB Virtual Sequence is derived from uvm_sequence and runs on the USB Virtual Sequencer. It controls and synchronizes the execution of Host sequences, Device sequences, and PHY sequences. The Virtual Sequence is responsible for generating complete USB 2.0 protocol scenarios such as enumeration, SETUP transactions, IN and OUT transactions, bulk transfers, interrupt transfers,

isochronous transfers, and error injection scenarios. It ensures proper coordination between Host Controller Agent, Device Controller Agent, and PHY Agent during protocol execution.

8.6 HOST CONTROLLER AGENT

The Host Controller Agent is an active agent responsible for generating and monitoring Host-side USB 2.0 protocol transactions. The Host Controller Agent consists of Host Sequencer, Host Driver, and Host Monitor.The Host Controller Agent communicates through the USB Host Interface. It generates token packets, data packets, and handshake packets according to the USB 2.0 protocol specification. The Host Controller Agent also monitors Host-side communication activity and protocol behavior.

8.6.1 HOST SEQUENCER

The Host Sequencer is derived from uvm_sequencer parameterized with the USB sequence item. It controls the flow of transaction items between the Host Sequences and Host Driver. The Host Sequencer receives randomized USB transactions such as IN, OUT, SETUP, DATA0, DATA1, ACK, NAK, and STALL packets from different Host sequences and forwards them to the Host Driver through TLM communication. It controls the sequencing and arbitration of Host-side USB transactions.

8.6.2 HOST DRIVER

The Host Driver is derived from uvm_driver and is responsible for converting transaction-level packet information into pin-level USB signaling through the USB Host Interface. The Host Driver receives sequence items from the Host Sequencer using the seq_item_port. It drives USB token packets, data packets, and handshake packets according to USB 2.0 timing requirements. The Host Driver also handles packet framing, CRC generation, reset signaling, and speed-specific timing for Low Speed, Full Speed, and High Speed operations. The Host Driver drives packet-level information onto the USB Host Interface signals.

8.6.3 HOST MONITOR

The Host Monitor is derived from uvm_monitor and passively observes Host-side USB protocol activity from the USB Host Interface. The Host Monitor captures pin-level USB signals and converts them into transaction-level packet information. It sends captured transactions to the Scoreboard and Subscriber using analysis ports. The Host Monitor observes token packets, data packets, handshake packets, packet timing, CRC fields, and protocol-level behavior. It provides protocol visibility and packet monitoring for Host-side verification.

8.6.4 HOST SEQUENCE

The Host Sequence is derived from uvm_sequence and is responsible for generating Host-side USB 2.0 transactions. The Host Sequence creates randomized sequence items and sends them to the Host Sequencer for execution. It generates different types of USB transactions such as SETUP packets, IN packets, OUT packets, DATA0/DATA1 packets, ACK/NAK/STALL handshake packets, and different transfer types including Control, Bulk, Interrupt, and Isochronous transfers. The Host Sequence controls Host-side protocol behavior and packet generation according to USB 2.0 specifications.

8.7 DEVICE CONTROLLER AGENT

The Device Controller Agent is an active agent responsible for generating and monitoring Device-side USB 2.0 protocol behavior. The Device Controller Agent consists of Device Sequencer, Device Driver , and Device Monitor. The Device Controller Agent communicates through the USB Devic e Interface. It generates Device responses, handshake packets, endpoint responses, and data packet transmissions according to the USB 2.0 protocol specification.

8.7.1 DEVICE SEQUENCER

The Device Sequencer is derived from uvm_sequencer parameterized with the USB sequence item. It controls the flow of Device-side transactions between Device sequences and the Device Driver. The Device Sequencer handles enumeration responses, endpoint transactions, data responses, and handshake generation such as ACK, NAK, and STALL packets. It forwards transaction items to the Device Driver using TLM communication.

8.7.2 DEVICE DRIVER

The Device Driver is derived from uvm_driver and is responsible for converting transaction-level packet information into pin-level USB signaling through the USB Device Interface. The Device Driver retrieves sequence items from the Device Sequencer and drives USB responses according to Host requests. It handles descriptor transmission, endpoint data transfers, handshake packet generation, and response timing according to USB 2.0 protocol specifications. The Device Driver drives Device-side packet information onto the USB Device Interface signals.

8.7.3 DEVICE MONITOR

The Device Monitor is derived from uvm_monitor and passively captures Device-side USB protocol activity from the USB Device Interface. The Device Monitor converts pin-level USB signaling into transaction-level packet information and sends captured transactions to the Scoreboard and Subscriber using analysis ports. It monitors Device responses, endpoint transactions, data packets, handshake packets, and enumeration behavior for protocol verification.

8.7.4 DEVICE SEQUENCE

The Device Sequence is derived from uvm_sequence and is responsible for generating Device-side USB 2.0 responses and transactions. The Device Sequence creates sequence items corresponding to Device responses such as descriptor responses, endpoint data transfers, ACK/NAK/STALL handshake packets, and enumeration-related responses. These sequence items are sent to the Device Sequencer for execution. The Device Sequence enables verification of different Device-side protocol scenarios and response behaviors according to USB 2.0 specifications.

8.8 PHY AGENT

The PHY Agent is responsible for PHY-level USB verification through the USB PHY Interface and USB UTMI Interface. The PHY Agent consists of PHY Sequencer ,PHY Driver, and PHY Monitor. The PHY Agent verifies low-level USB signaling behavior including bit-level transmission, synchronization patterns, NRZI encoding, bit stuffing, end-of-packet signaling, and line-state behavior according to the USB 2.0 specification.

8.8.1 PHY SEQUENCER

The PHY Sequencer is derived from uvm_sequencer and controls the flow of PHY-level transaction items between PHY sequences and the PHY Driver. It manages low-level signaling sequences, synchronization sequences, and PHY packet generation sequences.

8.8.2 PHY DRIVER

The PHY Driver is derived from uvm_driver and converts PHY-level transaction information into low-level USB signaling through the USB PHY/UTMI Interface. The PHY Driver handles NRZI encoding, bit stuffing, synchronization pattern generation, and packet transmission timing according to USB 2.0 PHY requirements. It drives PHY-level signals through the USB PHY Interface.

8.8.3 PHY MONITOR

The PHY Monitor is derived from uvm_monitor and passively observes PHY-level USB communication from the USB PHY/UTMI Interface. The PHY Monitor captures low-level USB signaling and converts it into transaction-level information. It monitors synchronization patterns, bit stuffing, NRZI encoding, packet boundaries, and line-state transitions. The captured PHY transactions are sent to the Scoreboard and Subscriber using analysis ports.

8.8.4 PHY SEQUENCE

The PHY Sequences are derived from `uvm_sequence` and are responsible for generating USB 2.0 PHY-side transactions for Low-Speed (LS), Full-Speed (FS), and High-Speed (HS) modes. These sequences create PHY sequence items corresponding to device-to-host transactions and protocol responses based on the selected USB speed mode. They help verify PHY-level communication, packet transmission, handshake responses, and speed-specific signaling behavior across different USB 2.0 operating modes.

8.9 SEQUENCE ITEM

The Sequence item consist of data fields required for generating the stimulus. In order to generate the stimulus randomly. We will be declaring data fields as rand variables. We will be also having Constraints to have control over the randomization.

8.10 SCOREBOARD

The Scoreboard is responsible for packet checking, protocol validation, and data comparison in the USB 2.0 VIP environment. It receives transaction packets from the Host Monitor and Device Monitor through analysis ports. The Host Monitor captures transmitted packet information from the Host side and sends Host TX packet data to the Scoreboard. Similarly, the Device Monitor captures received packet information from the Device side and sends Device RX packet data to the Scoreboard.

The Scoreboard compares the Host transmitted packet data with the Device received packet data to verify correct packet transmission and protocol behavior. In the same way, for Device-to-Host communication, the Device transmitted packet data captured by the Device Monitor is compared with the Host received packet data captured by the Host Monitor. The Scoreboard checks packet fields, packet type, address, endpoint information, payload data, handshake responses, and protocol-related information to ensure that transmitted and received packets match correctly.

The Scoreboard also performs protocol-level validation and reports mismatches, missing packets, unexpected responses, and data comparison failures during simulation.

8.11 SUBSCRIBER

The Subscriber is derived from uvm_subscriber and is responsible for functional coverage collection in the USB 2.0 UVM verification environment. The Subscriber receives transactions from both Host Monitor and Device Monitor through analysis ports and processes them inside the write() method. Based on the received transaction information, the Subscriber samples different covergroups for functional coverage collection.

Functional coverage includes token packets, data packets, handshake packets, transfer types (Control, Bulk, Interrupt, Isochronous), endpoint operations, speed modes (LS/FS/HS), PID types and protocol corner cases. The Subscriber collects coverage for both transmit and receive side transactions to ensure complete protocol-level verification.

The Subscriber helps measure the completeness and effectiveness of USB 2.0 protocol verification.

9.0 TYPES OF TRANSFERS

1. **Control Transfer**
2. **Bulk Transfer**
3. **Interrupt Transfer**
4. **Isochronous Transfer**

9.1 Control Transfer:

- Control transfer are intended to support configuration command status type communication flows between client and its function.
- The Default Control Pipe provides access to the USB device's configuration, status, and control information. Always goes through endpoint 0 (Control Endpoint).Supports IN and OUT directions.
  Example: Device enumeration, setting device address.

9.2 Bulk Transfers:

- Bulk data typically consists of larger amounts of data.
  Example: printers or scanners

9.3 Interrupt Transfer:

- A limited-latency transfer to or from a device is referred to as interrupt data.
  Example: Keyboard or Mouse

9.4 Isochronous Transfer:

- Used for continuous data streams requiring constant bandwidth. Example: Audio or video streaming from USB device.

10.0 ENUMERATION

10.1 USB Enumeration

- Process of detecting and configuringa USB device after connection.
- USB allows USB device to attach
- At this point USB device is in powered state and the port to which it is attached.
- The host issues reset command to the port, device enters default state.
- Host assigns unique address to the USB device and moving the device address state.
- Host reads device descriptor to determine max packet size.

10.2 Types of USB Descriptors

10.2.1 Device Descriptor – General info (VID, PID, USB version, max packet size).

10.2.2 Configuration Descriptor – Power requirements, it can support number of interfaces.

10.2.3 Interface Descriptor – Defines functionality(class/subclass)

10.2.4 Endpoint Descriptor – Endpoint address, type (Control, Bulk, Interrupt, Isochronous), max packet size.

11.0 USB PACKET TYPES

1 **Token Packet**

2 **Data Packet**

3 **Handshake Packet**

11.1 TOKEN PACKETS

There are three types of token packets:

- In
- Out
- Setup – used to begin control transfer

11.1.1 TOKEN PACKET FORMAT

SYNC (8/32 BIT)	PID (8 BIT)	ADDR (7 BIT)	ENDP (4 BIT)	CRC (5 BIT)	EOP

11.2 DATA PACKETS

There are two types of data packets each capable of transmitting up to 1024 bytes of data

- DATA0
- DATA1
- High speed mode defines another two data PIDs, DATA2 and MDATA.

11.2.1 DATA PACKET FORMAT

SYNC (8/32 BIT)	PID (8 BIT)	DATA	CRC (16 BIT)	EOP

11.3 HAND SHAKE PACKET

There are 3 types of hand packets which consist of the PID

- ACK
- NAK
- STALL

11.3.1 HAND SHAKE FPACKET FORMAT

SYNC (8 /32 BIT)	PID (8 BIT)	EOP

12.0 CONTROL TRANSFER

- Control transfer are typically used for command and status operations.
- They are essential to setup a USB device with all enumeration functions performed using control transfers.

The packet length of control transfers

- Low speed devices must be 8 bytes
- Full speed devices must be 8,16,32 and 64 bytes
- High speed devices must be 64 bytes

There are 3 stages

- Setup Stage
- Data Stage
- Status Stage

12.1 SET UP STAGE

When the request is sent, each stage consists of 3 packets

Setup token

Data packet (8 bytes)

Handshake packet (ACK)

Setup transaction:

SETUP TOKEN	DATA 0 (8 BYTES)	ACK HANDSHAKE

EXAMPLE

- Bmrequest - 1000 0000b
- Brequest – 06 - GET_DESCRIPTOR
- wvalue – (parameter value of device, configuration, endpoint, string, interface)
- windex – 01 - English – (language ID)
- Wlength – 0 – out, 1 – in (Direction 1)

12.2 DATA STAGE

- Optional, consists of one or multiple IN/OUT transfers
- Setup request indicates amount of data to be transmitted. Multiple data packets possible
- IN and OUT transfer.

Data transaction:

IN | OUT TOKEN	DATA1/DATA0	ACK NACK STALL HAND SHAKE

12.3 STATUS STAGE

- Reports the status of overall request and once varies due direction of transfer.
- Status reporting is always performed by the function.

Status transaction:

IN | OUT TOKEN	DATA1/ DATA0	ACK NACK STALL HAND SHAKE

13.0 BULK TRANSFER

- Bulk transfer defines maximum payload size support
- HIGH SPEED – 8,16,32,64 Bytes
- FULL SPEED – 512 Bytes
- LOW SPEED – Bulk endpoints are not supported

13.1 TYPES OF TRANSACTION

1. IN – TRANSACTION
2. OUT – TRANSACTION

13.2 IN – TRANSACTION

Host used to send a request data from bulk IN endpoint

- **HOST**– IN TOKEN
- **DEVICE** –DATA0 / DATA1
- **HOST** - ACK

IN TOKEN	DATA	ACK

13.3 OUT – TRANSACTION

Host sends data to bulk OUT endpoint

- **HOST** – OUT TOKEN
- **HOST** – DATA0 / DATA1
- **DEVICE** – ACK

OUT TOKEN	DATA	ACK

14.0 INTERRUPT TRANSFER

Interrupt transfer defines maximum payload size support

- LOW SPEED – 8 Bytes
- FULL SPEED – 64 Bytes
- HIGH SPEED - 1024 Bytes

14.1 TYPES OF TRANSACTION

1. IN – TRANSACTION
2. OUT – TRANSACTION

14.2 IN – TRANSACTION

- **HOST** – IN TOKEN
- **DEVICE** – DATA0 / DATA1
- **HOST** - ACK

IN TOKEN	DATA	ACK

14.3 OUT – TRANSACTION

- **HOST** – OUT TOKEN
- **HOST** – DATA0 / DATA1
- **DEVICE** – ACK

OUT TOKEN	DATA	ACK

15.0 ISOCHRONOUS TRANSFER

Isochronous transfer defines maximum payload size support

- LOW SPEED – Not supported
- FULL SPEED – 1023 Bytes
- HIGH SPEED - 1024 Bytes

15.1 TYPES OF TRANSACTION

1. IN – TRANSACTION
2. OUT – TRANSACTION

15.2 IN – TRANSACTION

- **HOST** – IN TOKEN
- **DEVICE** – DATA0 / DATA1

IN TOKEN	DATA	ACK

15.3 OUT – TRANSACTION

- **HOST** – OUT TOKEN
- **HOST** – DATA0 / DATA1

OUT TOKEN	DATA	ACK

16.0 Assertions:

SI No	Assertion name	Assertion Property	ETA	Status
1	usb2p0_p_dp_dm_not_both_high	property p_dp_dm_not_both_high; @(posedge clk) disable iff (reset) not (dp === 1'b1 && dm === 1'b1); endproperty	29/05/26	pass
2	usb2p0_p_rxvalid_with_rxactive	property p_rxvalid_with_rxactive; @(posedge utmi_clk) utmi_rxvalid |-> utmi_rxactive; endproperty	29/05/26	Pass
3	usb2p0_p_txvalid_txready_handshake	property p_txvalid_txready_handshake; @(posedge utmi_clk) utmi_txvalid |-> ##[0:5] utmi_txready; endproperty	29/05/26	pass
4	usb2p0_p_txdata_not_unknown	property p_txdata_not_unknown; @(posedge utmi_clk) utmi_txvalid |-> (!$isunknown(utmi_txdata)); endproperty	01/06/26	pass
5	usb2p0_p_rxdata_not_unknown	property p_rxdata_not_unknown; @(posedge utmi_clk) utmi_rxvalid |-> (!$isunknown(utmi_rxdata)); endproperty	01/06/26	Pass
6	usb2p0_valid_validh_wordif	property p_ls_fs_mode_check; @(posedge utmi_clk) (!utmi_txvalidh && !utmi_rxvalidh && utmi_txvalid && utmi_rxvalid) |-> (utmi_word_if == 1'b0); endproperty	01/06/26	Pass
7	usb2p0_valid_validh_wordif	property p_hs_mode_check; @(posedge utmi_clk) (utmi_txvalidh || utmi_rxvalidh) |-> (utmi_word_if == 1'b1); endproperty	02/06/26	pass
8	usb2p0_p_pid_check	property p_pid_check; @(posedge clk) disable iff (reset) $rose(pid_valid) |-> (pid_byte[7:4] == ~pid_byte[3:0]); endproperty	02/06/26	pass

9	usb2p0_p_pid_legal	property p_pid_legal;
		@(posedge utmi_clk) disable iff (reset)
		$rose(pid_valid) |->
		pid_byte[3:0] inside {
		4'hE, // OUT
		4'h6, // IN
		4'h2, // SETUP
		4'hC, // DATA0
		4'h4, // DATA1
		4'hD, // ACK
		4'h5, // NAK
		4'h1 // STALL
		};
		endproperty
			02/06/26	Pass

10	Usb2p0_in_transaction_order	property p_in_transaction_order; @(posedge utmi_clk) disable iff (reset) (pid_valid && pid_byte[3:0] == 4'h6) |-> ##[1:128] (device_pid_valid && device_pid_byte[3:0] == 4'h4) ##[1:128] (pid_valid && pid_byte[3:0] == 4'hd); endproperty	03/06/26	Pass
11	Usb2p0_out_transaction_order	property p_out_transaction_order; @(posedge utmi_clk) disable iff (reset) $rose(txvalid && pid_byte[3:0]==4'he) |-> ##[1:256] (pid_valid && pid_byte[3:0]==4'h4) ##[1:256] (device_pid_valid && device_pid_byte[3:0]==4'hd); endproperty	03/06/26	Pass

12	Usb2p0_setup_uses_data0	property p_setup_uses_data0;
		@(posedge utmi_clk) disable iff (reset)
		$rose(txvalid && pid_byte[3:0] == 4'h2) |->
		##[1:256] (data_valid || pid_byte[3:0] == 4'hc) //
		DATA0
		##[1:256] (device_pid_valid &&
		device_pid_byte[3:0]==4'hd);
		endproperty
			03/06/26	Pass

13	usb2p0_p_inter_packet_gap	property p_inter_packet_gap; @(posedge clk) disable iff (reset) $fell(pkt_active) |-> ##[10:$] $rose(pkt_active); endproperty	03/06/26	pass
14	usb2p0_p_no_handshake_after_token_direct	property p_no_handshake_after_token_direct; @(posedge clk) disable iff (reset) $rose(token_valid) |-> !handshake_valid[*1:5]; endproperty	04/06/26	Pass
15	usb2p0_p_linestate_known	property p_linestate_known; @(posedge utmi_clk) $rose(utmiotg_vbusvalid) |-> ##[1:10] (!$isunknown(utmi_linestate)); endproperty	04/06/26	Pass

16	usb2p0_p_vbus_valid	property p_vbus_valid; @(posedge utmi_clk) disable iff(reset) $rose(utmiotg_vbusvalid) |-> ##[1:10] utmisrp_bvalid; endproperty	04/06/26	Pass

Table No.1 Assertion

17.0 Coverage

SI.NO	cover_point_name	Bins

1	USB_PID_CP	bit [DATA_WIDTH-1:0] pidsample_host_tx;
		
		USB_PID_CP : coverpoint pidsample_host_tx {
		Bins setup_pid_htx = {8'hD2};
		bins data0_pid_htx = {8'h3C};
		bins ack_pid_htx = {8'h2D};
		bins in = {8'h96};
		bins out = {8'h1E};
		bins data1 = {8'hB4};
		}

2	USB_ADDR_CP	bit [7:0]ADDRESS;
		
		ADDR: coverpoint usb_seq_item.addr{
		bins device_addr0 = {0};
		ignore_bins others = {[0:127]} with (item != 0);
		}

3	USB_ENDPOINT_CP	bit [3:0] endp;
		
		ENDPOINT : coverpoint usb_seq_item.endp {
		bins control_ep = {0};
		bins bulk_in_ep = {1};
		bins bulk_out_ep = {2};
		bins interrupt_in_ep = {3};
		bins interrupt_out_ep = {5};
		bins iso_in_ep = {6};
		bins iso_out_ep = {7};
		ignore_bins others = {[8:15]};
		}

4	USB_BREQUEST_CP	BREQUEST: coverpoint usb_seq_item.bRequest {
		bins get_status = {8'h00};
		bins clear_feature = {8'h01};
		bins set_feature = {8'h03};
		bins set_address = {8'h05};
		bins get_descriptor = {8'h06};
		bins get_config = {8'h08};
		bins set_config = {8'h09};
		bins get_interface = {8'h0A};
		bins set_interface = {8'h0B};
		ignore_bins others = {[0:255]} with (!(item inside {
		8'h00,8'h01,8'h03,8'h05,8'h06,8'h07,8'h08,8'h09,8'h0A,8'h0B
		}));
		}

5	USB_BMREQUESTTYPE_CP	BMREQUESTTYPE : coverpoint usb_seq_item.bmRequestType { bins std_out_device = {8'h00}; bins std_device ={8'h01}; bins std_in_device = {8'h80}; ignore_bins = {[0:255]} with (!(item inside {8'h00,8'h01,8'h80,8'h81})); }
6	USB_WVALUE_CP	WVALUE : coverpoint usb_seq_item.wValue { bins zero = {16'h0000}; bins one = {16'h0001}; ignore_bins = {[0:65535]} with (!(item inside {16'h0000,16'h0001})); }
7	USB_WINDEX_CP	WINDEX : coverpoint usb_seq_item.wIndex { bins wzero = {16'h0000}; bins wone = {16'h0001}; bins val_81 = {16'h0081}; ignore_bins = {[0:65535]} with (!(item inside {16'h0000,16'h0001,16'h0081}));
8	USB_WLENGTH_CP	WLENGTH : coverpoint usb_seq_item.wLength { bins zero = {16'h0000}; bins two = {16'h0002}; bins desc = {16'h0012}; ignore_bins = {[0:65535]} with (!(item inside {16'h0000,16'h0002,16'h0012})); }
9	USB_BLENGTH_CP	BLENGTH : coverpoint usb_seq_item.blength { bins device_len = {8'h12}; ignore_bins = {[0:255]} with (item != 8'h12); }
10	USB_DESC_TYPE_CP	DESC_TYPE : coverpoint usb_seq_item.bdescriptors_type { bins device_desc = {8'h01}; ignore_bins others = {[0:255]} with (item != 8'h01); }
11	USB_BCD_USB_CP	BCD_USB : coverpoint usb_seq_item.bcd_usb { bins usb_1 = {16'h0110}; ignore_bins others = {[0:65535]} with (item !=16'h0110); }
12	USB_DEVICE_CLASS_CP	DEVICE_CLASS : coverpoint usb_seq_item.bDevice_class { bins bdeviceclass = {8'h00}; ignore_bins = {[0:255]} with (item != 8'h00); }
13	USB_DEVICE_SUBCLASS_CP	DEVICE_SUBCLASS : coverpoint usb_seq_item.bDevice_subclass { bins subclass = {8'h00}; ignore_bins s = {[0:255]} with (item != 8'h00); }

14	USB_DEVICE_PROTOCOL_CP	DEVICE_PROTOCOL : coverpoint
		usb_seq_item.bDevice_protocol {
		bins protocol = {8'h00};
		ignore_bins= {[0:255]} with (item != 8'h00);

15	USB_MAX_PACKET_CP	MAX_PACKET : coverpoint
		sb_seq_item.bMax_packetsize {
		bins packetsize = {8'h40};
		ignore_binss = {[0:255]} with (item != 8'h40);
		}

16	USB_VENDOR_ID_CP	VENDOR_ID : coverpoint usb_seq_item.idvendor {
		bins vendor_781 = {16'h0781};
		ignore_bins = {[0:65535]} with (item != 16'h0781);
		}

17	USB_PRODUCT_ID_CP	PRODUCT_ID : coverpoint usb_seq_item.idproduct {
		bins product_5567 = {16'h5567};
		ignore_bins = {[0:65535]} with (item != 16'h5567);
		}

18	USB_DEVICE_BCD_CP	DEVICE_BCD : coverpoint usb_seq_item.bcdDevice {
		bins dev_ver = {16'h0126};
		ignore_bins = {[0:65535]} with (item != 16'h0126);
		}

19	USB_MANUF_STR_CP	MANUF_STR : coverpoint usb_seq_item.imanufacture {
		bins mfg = {8'h01};
		ignore_bins = {[0:255]} with (item != 8'h01);
		}

20	USB_PROD_STR_CP	PROD_STR : coverpoint usb_seq_item.iproduct {
		bins product = {8'h02};
		ignore_bins = {[0:255]} with (item != 8'h02);
		}

21	USB_SERIAL_STR_CP	SERIAL_STR : coverpoint usb_seq_item.iserial_number {
		bins serial = {8'h03};
		ignore_bins = {[0:255]} with (item != 8'h03);
		}

22	USB_NUM_CONFIG_CP	NUM_CONFIG : coverpoint usb_seq_item.bNum_configuration {
		bins cfg = {8'h01};
		ignore_bins others = {[0:255]} with (item != 8'h01);
		}

23	USB_TX_VALID_CP	TX_VALID : coverpoint usb_seq_item.tx_valid;
24	USB_TX_VALIDH_CP	TX_VALIDH : coverpoint usb_seq_item.tx_validh;
25	USB_RX_VALID_CP	RX_VALID : coverpoint usb_seq_item.rx_valid;

26	USB_RX_VALIDH _CP	RX_VALIDH : coverpoint usb_seq_item.rx_validh;
27	USB_WORD_IF_CP	WORD_IF : coverpoint usb_seq_item.word_if;

28	USB_OP_MODE_CP	OP_MODE : coverpoint usb_seq_item.op_mode {
		bins hs_only = {2'b00};
		ignore_bins others = {[0:3]} with (item != 2'b00);
		}

29	USB_TERM_SELECT_CP	TERM_SELECT : coverpoint usb seq_item.term_select;
30	USB_XCVR_SELECT_CP	XCVR_SELECT : coverpoint usb_seq_item.xcvr_select;
31	USB_SUSPEND_N_CP	SUSPEND_N : coverpoint usb_seq_item.suspend_n;
32	USB_FSLS_SERIALMODE_CP	FSLS_SERIALMODE : coverpoint usb_seq_item.fsls_serialmode;
33	USB_FSLS_LOW_POWER_CP	FSLS_LOW_POWER : coverpoint usb_seq_item.fsls_low_power;
34	USB_OTG_DPPULLDOWN_CP	OTG_DPPULLDOWN : coverpoint usb_seq_item.otg_dppulldown;
35	USB_OTG_DMPULLDOWN_CP	OTG_DMPULLDOWN : coverpoint usb_seq_item.otg_dmpulldown;
36	USB_FFFUTMI_VALID_CROSS_CP	UTMI_VALID_CROSS : cross TX_VALID, RX_VALID;

Table No.1 Coverage Table

18.0 Testcases Table:

S.No	Testcase name	ETA	Status
1	usb2p0_pm_power_connect	11/19/2025	Pass
2	usb2p0_pm_power_disconnect	11/19/2025	Pass
3	usb2p0_ls_speed_detect	11/20/2025	Pass
4	usb2p0_fs_speed_detect	11/20/2025	Pass
5	usb2p0_hs_speed_detect	11/20/2025	Pass
6	USB2p0_LS_CT_clear_feature_test	11/24/2025	Pass
7	USB2p0_FS_CT_clear_feature_test	11/24/2025	Pass
8	USB2p0_HS_CT_clear_feature_test	1/22/2026	Pass
9	USB2p0_LS_CT_set_address_test	11/26/2025	Pass
10	USB2p0_FS_CT_set_address_test	11/26/2025	Pass
11	USB2p0_HS_CT_set_address_test	1/27/2026	Pass
12	USB2p0_LS_CT_set_configuration_test	11/28/2025	Pass
13	USB2p0_FS_CT_set_configuration_test	11/28/2025	Pass
14	USB2p0_HS_CT_set_configuration_test	1/30/2026	Pass
15	USB2p0_LS_CT_set_feature_test	12/2/2025	Pass
16	USB2p0_FS_CT_set_feature_test	12/2/2025	Pass
17	USB2p0_HS_CT_set_feature_test	2/4/2026	Pass
18	USB2p0_LS_CT_set_interface_test	12/5/2025	Pass
19	USB2p0_FS_CT_set_interface_test	12/5/2025	Pass
20	USB2p0_HS_CT_set_interface_test	2/9/2026	Pass
21	USB2p0_LS_CT_get_descriptor_test	12/8/2025	Pass
22	USB2p0_FS_CT_get_descriptor_test	12/8/2025	Pass
23	USB2p0_HS_CT_get_descriptor_test	2/13/2026	Pass
24	usb2p0_CT_LS_get_status_test	12/12/2025	Pass
25	usb2p0_CT_FS_get_status_test	12/12/2025	Pass
26	usb2p0_CT_HS_get_status_test	2/18/2026	Pass
27	usb2p0_CT_LS_get_config_test	12/16/2025	Pass
28	usb2p0_CT_FS_get_config_test	12/16/2025	Pass
29	usb2p0_CT_HS_get_config_test	2/23/2026	Pass
30	usb2p0_CT_LS_get_interface_test	12/19/2025	Pass
31	usb2p0_CT_FS_get_interface_test	12/19/2025	Pass
32	usb2p0_CT_HS_get_interface_test	2/26/2026	Pass
33	usb2p0_LS_INTR_IN_transfer	12/24/2025	Pass
34	usb2p0_FS_INTR_IN_transfer	12/24/2025	Pass
35	usb2p0_HS_INTR_IN_transfer	3/2/2026	Pass
36	usb2p0_LS_INTR_OUT_transfer	1/7/2026	Pass
37	usb2p0_FS_INTR_OUT_transfer	1/7/2026	Pass

38	usb2p0_HS_INTR_OUT_transfer	3/5/2026	Pass
39	usb2p0_FS_BULK_IN_transfer	1/12/2026	Pass
40	usb2p0_HS_BULK_IN_transfer	3/9/2026	Pass
41	usb2p0_FS_BULK_OUT_transfer	1/12/2026	Pass
42	usb2p0_HS_BULK_OUT_transfer	3/12/2026	Pass
43	usb2p0_FS_ISO_IN_transfer	1/16/2026	Pass
44	usb2p0_HS_ISO_IN_transfer	3/16/2026	Pass
45	usb2p0_FS_ISO_OUT_transfer	1/16/2026	pass
46	usb2p0_HS_ISO_OUT_transfer	3/20/2026	Pass
47	USB2p0_LS_CT_clear_feature_error_test	3/23/2026	Pass
48	USB2p0_FS_CT_clear_feature_error_test	3/23/2026	Pass
49	USB2p0_HS_CT_clear_feature_error_test	5/13/2026	Pass
50	USB2p0_LS_CT_set_address_error_test	3/27/2026	Pass
51	USB2p0_FS_CT_set_address_error_test	3/27/2026	Pass
52	USB2p0_HS_CT_set_address_error_test	5/13/2026	Pass
53	USB2p0_LS_CT_set_configuration_error_test	3/31/2026	Pass
54	USB2p0_FS_CT_set_configuration_error_test	4/1/2026	Pass
55	USB2p0_HS_CT_set_configuration_error_test	5/13/2026	Pass
56	USB2p0_LS_CT_set_feature_error_test	4/6/2026	Pass
57	USB2p0_FS_CT_set_feature_error_test	4/9/2026	Pass
58	USB2p0_HS_CT_set_feature_error_test	5/15/2026	Pass
59	USB2p0_LS_CT_set_interface_error_test	4/13/2026	Pass
60	USB2p0_FS_CT_set_interface_error_test	4/13/2026	Pass
61	USB2p0_HS_CT_set_interface_error_test	5/15/2026	Pass
62	USB2p0_LS_CT_get_descriptor_error_test	4/16/2026	Pass
63	USB2p0_FS_CT_get_descriptor_error_test	4/16/2026	Pass
64	USB2p0_HS_CT_get_descriptor_error_test	5/19/2026	Pass
65	usb2p0_CT_LS_get_status_error_test	4/20/2026	Pass
66	usb2p0_CT_FS_get_status_error_test	4/20/2026	Pass
67	usb2p0_CT_HS_get_status_error_test	5/19/2026	Pass
68	usb2p0_CT_LS_get_config_error_test	4/23/2026	Pass
69	usb2p0_CT_FS_get_config_error_test	4/23/2026	Pass
70	usb2p0_CT_HS_get_config_error_test	5/19/2026	Pass
71	usb2p0_CT_LS_get_interface_error_test	4/27/2026	Pass
72	usb2p0_CT_FS_get_interface_error_test	4/27/2026	Pass
73	usb2p0_CT_HS_get_interface_error_test	5/19/2026	Pass
74	usb2p0_LS_INTR_IN_error_transfer	4/29/2026	Pass
75	usb2p0_FS_INTR_IN_error_transfer	4/29/2026	Pass
76	usb2p0_HS_INTR_IN_error_transfer	5/20/2026	Pass
77	usb2p0_LS_INTR_OUT_transfer	5/5/2026	Pass
78	usb2p0_FS_INTR_OUT_transfer	5/5/2026	Pass

79	usb2p0_HS_INTR_OUT_transfer	5/20/2026	Pass
80	usb2p0_FS_BULK_IN_error_transfer	5/7/2026	Pass
81	usb2p0_HS_BULK_IN_error_transfer	5/21/2026	Pass
82	usb2p0_FS_BULK_OUT_error_transfer	5/7/2026	Pass
83	usb2p0_HS_BULK_OUT_error_transfer	5/21/2026	Pass
84	usb2p0_FS_ISO_IN_error_transfer	5/11/2026	Pass
85	usb2p0_HS_ISO_IN_error_transfer	5/22/2026	Pass
86	usb2p0_FS_ISO_OUT_error_transfer	5/11/2026	Pass
87	usb2p0_HS_ISO_OUT_error_transfer	5/22/2026	Pass
88	usb2p0_LS_crc5_error_test	5/19/2026	pass
89	usb2p0_FS_crc5_error_test	5/20/2026	pass
90	usb2p0_HS_crc5_error_test	5/21/2026	pass
91	usb2p0_LS_crc16_error_test	5/19/2026	pass
92	usb2p0_FS_crc16_error_test	5/20/2026	pass
93	usb2p0_HS_crc16_error_test	5/21/2026	pass
94	usb2p0_LS_CT_random_testcase	5/21/2026	Pass
95	usb2p0_FS_CT_random_testcase	5/21/2026	Pass
96	usb2p0_HS_CT_random_testcase	5/21/2026	Pass
97	Usb2p0_LS _interrupt_in_ random _testcase	5/22/2026	Pass
98	Usb2p0_FS _interrupt_in_ random _testcase	5/22/2026	Pass
99	Usb2p0_HS _interrupt_in_ random _testcase	5/22/2026	Pass
100	Usb2p0_LS _interrupt_out_ random _testcase	5/22/2026	Pass
101	Usb2p0_FS _interrupt_out_ random _testcase	5/22/2026	Pass
102	Usb2p0_HS _interrupt_out_ random _testcase	5/22/2026	Pass
103	usb2p0_FS_bulk_in_random_testcase	5/25/2026	Pass
104	usb2p0_HS_bulk_in_random_testcase	5/25/2026	Pass
105	usb2p0_FS_bulk_out_random_testcase	5/25/2026	Pass
106	usb2p0_HS_bulk_out_random_testcase	5/25/2026	Pass
107	usb2p0_FS_iso_in_random_testcase	5/26/2026	Pass
108	usb2p0_HS_iso_in_random_testcase	5/26/2026	Pass
109	usb2p0_FS_iso_out_random_testcase	5/26/2026	Pass
110	usb2p0_HS_iso_out_random_testcase	5/26/2026	Pass

Table No.1 Testcase Table